🌿 Branches (Trabalhando em paralelo sem quebrar o projeto)

Aqui você aprende a **não destruir seu próprio código** enquanto evolui.

🧠 O que é uma branch?

👉 Uma **branch (ramo)** é uma linha paralela do seu projeto.

É como criar uma cópia do projeto para testar coisas sem mexer no original.

🌳 Visual simples

```
main (principal)
  │
  ├── nova-funcionalidade
  ├── correção-bug
  └── teste
```

💡 Ideia central

Você nunca trabalha direto no código principal.

⚙️ 1. `git branch` — ver branches

```
git branch
```

🔍 Exemplo

```
* main
  login
  teste
```

👉 * = branch atual

➕ 2. Criar branch

```
git branch nova-branch
```

🔧 Exemplo real

```
git branch login
```

👉 cria uma nova linha de desenvolvimento

🔄 3. Trocar de branch

```
git switch nome
```

ou:

```
git checkout nome
```

🔧 Exemplo

```
git switch login
```

👉 agora você está trabalhando nessa branch

🚀 4. Criar e trocar ao mesmo tempo

```
git switch -c nova-branch
```

💧 Exemplo

```
git switch -c sistema-pagamento
```

👉 cria e já entra nela

🧠 O que muda na prática?

Cada branch tem:

- seus commits
- suas mudanças
- sua evolução

💧 Exemplo real (seu cenário)

Você está criando um sistema de delivery:

❌ Forma errada

Trabalhar direto na `main`

👉 risco de quebrar tudo

☑️ Forma certa

```
git switch -c login
```

Trabalha no login

```
git switch -c pagamento
```

Trabalha no pagamento

🔀 5. `git merge` — juntar branches

📌 Comando:

```
git merge outra-branch
```

🔧 Exemplo

```
git switch main
git merge login
```

👉 junta o login no projeto principal

🧠 Visual

Antes:

```
main: A — B
login:     C — D
```

Depois:

```
main: A — B — C — D
```

⚠️ 6. Conflitos (parte mais importante)

👉 acontece quando duas branches mudam a mesma coisa

🔥 Exemplo

Arquivo:

```
preço = 10
```

Branch A muda:

```
preço = 20
```

Branch B muda:

```
preço = 30
```

👉 Git não sabe qual escolher

💥 Resultado:

```
CONFLICT
```

🧠 Como resolver?

Git mostra assim:

```
<<<<<<< HEAD
preço = 20
=======
preço = 30
>>>>>>> branch
```

✔️ Você escolhe:

```
preço = 30
```

Depois:

```
git add .
git commit
```

🗑️ 7. Deletar branch

```
git branch -d nome
```

⚠️ Forçar exclusão:

```
git branch -D nome
```

🧠 Mentalidade profissional

Branches servem para:

- testar ideias
- desenvolver funcionalidades
- corrigir bugs
- trabalhar em equipe

🧬 Fluxo profissional (muito importante)

```
main (estável)
   |
   ├── feature/login
   ├── feature/pagamento
   └── bugfix/erro-login
```

🚀 Exemplo completo (real)

```
git init

git switch -c login
echo "login" > app.txt
git add .
git commit -m "cria login"

git switch main
```

```
git merge login
```

🧠 O que você aprendeu aqui

Você agora sabe:

- trabalhar em paralelo
- evitar quebrar código
- organizar desenvolvimento
- preparar projeto profissional

🔥 Próximo nível (AGORA FICA SÉRIO)

👉 **Rebase vs Merge (nível avançado de Git)**

Isso é o que separa:

- iniciante
- intermediário
- engenheiro de verdade